

Symbol Store Architecture

1 Declaring symbolic expressions

1.1 shared API

proposition for the interface. to be discussed/adjusted

```
class Expression {
public:
    // compile-time type deduction of the arguments
    Expression(std::string const& head, Arguments... arguments);

    // for dynamic expression construction
    // wrap any data type as an Expression (Expression, named Symbol or unnamed value)
    // so we can use arguments which are a mix bag of values or expressions
    Expression(std::string const& head, std::vector<Expression> & arguments);

    template<T> T get();

    std::string toString(); // stringified value or expression (if not evaluated)
}

class Symbol : public Expression {
public:
    // named symbol (undefined at construction)
    // need to be explicitly call as Symbol("name")
    Symbol(std::string const& name);

    // unnamed symbol, as a wrapper for a value of any supported type
    template<typename T>
    explicit Symbol(T value);

    template<typename T>
    define(T value); // T is any supported data type or an expression
}

Expression evaluate(Expression &); // return defined symbol if evaluated
```

```
// expression if not evaluated
```

1.2 Additional layers of expressivity

Even though the implementation is in C++, the expressions are also intended to be used in higher-level languages such as Python. For the sake of readability, we introduce additional layers (2 and 3) beside the base expression API defined previously (1).

1. pure expression	e.g.	<code>evaluate(Expression{"function", arg1, arg2})</code>
2. static function	e.g.	<code>function(arg1, arg2)</code>
3. object-oriented	e.g.	<code>arg1.function(arg2)</code>

These two higher-level API are intended to be implemented as wrappers for the lower-level implementation. They only need to support built-in operators (they can be generated along with the definition of the built-in operators). We can potentially add the common built-ins operators to the shared API.

To be discussed: do all the backends need to implement those additional functions/methods?

2 Storing symbolic expressions

2.1 Symbolic database

Expressions are stored as tuples in a relational database engine.

We need to support basic operations for updating tuples in the database:

`insert, delete, update`

Additionally we need to support operators to create, modify and access a table and table properties:

`create_table, delete_table, add_column, remove_column, table, column_index, column_name`

For simplicity and flexibility, we consider a table and any transaction result as an expression. It allows us to encapsulate a table or any transaction as a generic object which can also be used inside an expression.

We can then write any of the examples below as valid expressions:

```
Expression("create_table", "T1")
Expression("create_column", Expression("table", "T1"), "ColA", "number")
Expression("insert", Expression("table", "T1"), {1, "name", Expression("+", 1, 2)})
Expression("project", Expression("table", "T1"), {"ColA"})
```

2.2 Serialization

Beside creating the database from code, we should additionally support writing/loading to disk:

- write/load binary files from the tuple raw data stored in database (specific to each backend)
- load schema from file (json etc). can be common code to every backend as far as we have the same API for creating tables/columns
- load tuples from file (tbl, csv etc). can be common code to every backend as far as we have the same API for inserting tuples

2.3 Implementation

If it is possible to share the storage manager layer with every backend and expose only functions to extract data for the query processor, it will allow to keep the whole storage data structure / storage operators / serialization implemented only once.

To be confirmed if this approach is possible, it depends on the needs for every backend implementation.

If confirmed, need to define the interface for the storage manager

3 Evaluating symbolic expressions

We are not evaluating only single expressions. We want to support **bulk evaluation**.

- We should be able to evaluate efficiently all the expressions in a column for a table.

The user needs some control on when to do the evaluation

- **lazy evaluation**: we don't automatically evaluate at insert, and until the field value is needed by an operation
- additionally, we can explicitly call `evaluate()` on a whole table or table's column
- we also need a special query operator or criteria to find tuples containing yet un-evaluated expressions

The user needs some control on what to do with the result of the evaluation

To be confirmed here what makes sense between

- A: By default, calling `evaluate` on a table returns evaluated temporary tuples and leaves the stored expression untouched. Evaluations can be made definitive in the table by using `update` or `delete/insert`
- B: Calling `evaluate` modifies directly the field stored in the database (permanently evaluated).

4 Built-in operators

To be discussed: which ones of those operators are the core operators to be supported on every backend

4.1 Operators implemented for each backend

The following categories of operators are specific to the underline implementation of the backend query processor. They will have to be implemented separately for each backend.

4.1.1 Query operators

```
select(table, column, predicate)
project(table, columns[])
aggregate(table, column, operator)
hash(table, column)
probe(hashmap, table, column, predicate)
```

4.1.2 Table definition

```
create_table(tablename)
table(table_name)
add_column(table, columnname, datatype)
remove_column(table, column)
```

4.1.3 Table manipulation

```
insert(table, tuple[])
delete(table, column, predicate)
update(table, column, value, predicate)
```

4.2 Operators independent from backend implementation

The following categories of operators are not directly related with the backend implementation. They can potentially be implemented as a common code for all backends (if relevant to the backend and if the backend interface to add built-in functions permits).

4.2.1 Arithmetic, comparison, logic

```
add, sub, mul, div
<, >, <=, >=, ==, !=
and, or, not
```

4.2.2 Fetching

Pull-based data transferred from network

- It should allow to request data from public API
- **distributed processing**: it should allow to request data from other clients holding the same database
- It should be optimised for grouped requests

`fetch(url, command, fieldname)`

to be discussed: method to be able to group requests will be affected by the fetch behaviour. non-blocking? blocking + caching?

4.2.3 Interpolation

Linear interpolation of value Y at position X (axis X being typically the time series) based on neighbour points `prev(X,Y)` and `next(X,Y)`

`interpolate(prevX, prevY, nextX, nextY, targetX)`
`interpolate(columnX, columnY, valueX)`

It might be possible to express it purely as a compound expression from other built-in operators